d need to run the
application and tests.

The above option may also make use of Dev Containers.

User confirmation for commands

Another option for limiting the risk is to require the user to confirm every command the agent executes before it runs the command. We will likely be implementing this as an option anyway but given the desire for efficient development of larger workflows it might limit the efficiency of the tool if it needs to execute a lot of commands to finish a task.

We may also consider a hybrid approach where there a set of user-defined allowlisted commands (e.g. ls and cat) which allow the agent to read and learn about a project without the user needing to confirm. This approach may not solve all needs though where the user may want to allowlist commands like rspec which then effectively still allow for arbitrary code execution as the agent can put whatever they want in the spec file.

Workflow UI

The Workflow UI will need to be available at least in the following places:

1. In GitLab Rails web UI
1. In our editor extensions

The fact that we'll need multiple UIs and also as described above we have multiple execution environments for Workflow executor have led to the following decisions.

How do we package and run the local web UI

We will build the majority of data access related to our local IDE UI into the GitLab Language Server
to maximize re-use across all our editor extensions. We will also employ a mix of webviews rendered in the IDE and served by the LSP as well as native IDE UI elements. Where it doesn't considerably limit our user experience we'll opt to build the interface into a web page served from the LSP and then rendered in the IDE as a web view because this again maximises re-use across all our editor extensions.

How does the web UI reflect the current state live

The Workflow service regularly saves its state to GitLab
Rails. The UI receives real-time updates through GraphQL
subscriptions, automatically refreshing as new data arrives.

Given that the user may be running the Workflow executor locally which may be seeing some of the state as it happens it might be reasonable to want to

just live render the in-memory state of the running workflow process. We may choose this option deliberately for latency reasons, but we need to be careful to architect the frontend and Workflow executor as completely decoupled because they will not always be running together. For example users may trigger a workflow locally which runs in GitLab CI or they may be using the web UI to interact with and re-run a workflow that was initiated locally.

As such we will generally prefer not to have direct interaction between the UI and executor, but instead all communication should be happening via GitLab. Any exceptions to this might be considered case by case, but we'll need clear API boundaries to allow the functionality to easily be changed to consume from GitLab for the reasons described.

States

UI states are going to be driven by the server to ensure consistency and no race condition between UI state changes and API responses. This will simplify keeping data and action in sync. All possible workflow states can be seen in the WorflowStatusEnum.

New

The first UI state is when the user needs to submit a goal. This is only a client-side concern and does not translate to any server-side state.

Not started / created

After starting a workflow with a user-defined goal, a workflow is created which translates to LangGraph creating an empty checkpoint. On the API side, the workflow will be created with the NOTSTARTED state, which means that a goal has been sent, but no plan has been returned from LangGraph yet. This is the point where the client needs to start subscribing to DuoWorkflowEvent.

Planning

Once in planning, we need to start streaming Workflow responses back to the client. We are getting the checkpoint field from the DuoWorkflowEvent subscription which is a JsonString that represents LangGraph raw response. For the first iteration, the client will rely solely on this JSON string to get all new data and drive the UI, which means that we need to parse the JSON and account for possible parsing errors. It also means having a very tight coupling between the UI and LangGraph, so this parsing should be abstracted away in its own functionality so that it can be easily removed.

Important: For the first iteration, there is no way to stop or pause a Workflow and this mean that once the goal has been submitted, we will not render any button or UI interactions. The UI will "move on its own" where for example, once the PLANNING phase is over, we will automatically open the EXECUTING panel and start rendering the right information. Users do not need to confirm or interact and the Workflow should reach its final state on its own.

Executing

What should drive the progress bar is the subscription to workflowEventsUpdated and getting checkpoints streamed back to the client so that we know what progress was made in the execution and what step is currently being executed. This is where the user watches steps and progress being made.

Completed

Once all of the events are done streaming, the API should return the COMPLETED state and we can show the finished screen.

Future iterations

As of writing this documentation, there are no PENDING state as this is currently not planned for the first version of the Workflow, but it's worth keeping in mind for future iterations.
This mean that the user cannot pause execution and/or we cannot have users interact during execution. The Workflow will eventually be interactable and these interactions should be added to this documentation.

Checkpoints

Checkpoints are built on the server-side and serve as a way to resume execution from certain points. There are some interactions between checkpoints and the UI that are foundational to the experience. Checkpoint should:

- Be streamed to the client for real time update.
- Contain the current state.
- Contain the current step being executed.

Ability to load workflow from checkpoints

If a Workflow cannot reach the COMPLETED state before it stops executing, then the user should have the ability to start again only from the latest checkpoint. From a UI perspective, this mean that we need to support multiple workflow because otherwise we cannot know which workflow currently exists. This should be doable by fetching duoWorkflowWorkflows and rendering all existing workflows. For the IDEs, this mean rendering existing workflows in the sidebar.

Then when selecting a workflow, fetch information about the current workflow with how to fetch an existing workflow. Here is what should happen based on states:

- NONE: The user will not have any state preserved, meaning that any goal typed out, but not submitted will not be kept.
- NOT STARTED: resends the initial request to generate a plan and shows the loading state.
- PLANNING: loading the workflow should show existing proposed plan.
- EXECUTING: Resume from the right step. In future iterations. the Workflow should probably be "stopped/paused" and then the user could resume. For this added functionality, we need the PENDING state to exists.
- COMPLETED: Show the result of the workflow after its completion.

Workflow agent's tools

Workflow agents are, in a simplified view, a pair of: prompt and LLM.
By this definition, agents on their own are not able to interact with the outside world, which significantly limits the scope of work that can be automated. To overcome this limitation, agents are being equipped with tools.

Tools are functions that agents can invoke using the function calling LLM feature.
These functions perform different actions on behalf of the agent. For example, an agent might be equipped with a tool (function)
that executes bash commands like ls or cat and returns the result of those bash commands back to the agent.

The breadth of the tool set available to agents defines the scope of work that can be automated. Therefore, to
set up the Workflow feature for success, it will be required to deliver a broad and exhaustive tool set.

Foreseen tools include:

1. Tools to execute bash commands via the Workflow executor.
1. Tools to manipulate files (including reading and writing to files).
1. Tools to manipulate Git VCS.
1. Tools to integrate with the GitLab HTTP API.

The fact that the Workflow service is going to require Git and GitLab API tools entails that the Workflow service
must have the ability to establish an SSH connection and make HTTP requests to the GitLab instance. This ability can be granted directly to the Workflow service or can be provided via the Workflow executor if a direct connection between the Workflow service and a GitLab instance is not possible due to a firewall or network partition.

Tools permissions and approval system

Equipping agents with tools comes with different risk factors. For example, read tools might cross boundaries between confidential and public data if applied incorrectly, and tools that integrate directly with Duo Workflow Executor host bash terminal can open a whole range of severe consequences when agents make mistakes or get tricked into performing malicious actions. In order to limit the negative impact of different tools, a tool approval system has been implemented, granting users the ability to limit the set of available tools for any given workflow run, as well
as enforce agents to seek user approval before certain tools are executed.

The tools approval system is based on a bucket approach, where tool buckets are named agent privileges. Each bucket outlines a subset of all implemented tools
which users can make fully available or conditionally available, following the process outlined in the next section. Agent privileges are defined in the toolsregistry within Duo Workflow Service and are reflected in the Workflow GitLab Rails model.

How agent tool set is being defined for each workflow run

1. An engineer defines an agent's tool set during workflow implementation by listing all

possible tools that could be granted to a model.
1. Upon creation of a workflow run, a request is made to workflow's GitLab API endpoint with agentprivileges that limits the complete scope of the tool set defined in step 1 by the engineer into a subset constrained by user-granted agent privileges. In addition, that API request may include preapprovedagentprivileges which allow agents to use tools from listed buckets without asking for approval.

How tools approvals are being enforced

Tools approval verification happens between an agent's node that produces LLM-generated function calls and the ToolExecutor node that runs LLM's function calls.
Before the ToolExecutor node is triggered, all pending function calls are reviewed against an allow list that is defined with preapprovedagentprivileges following
the process outlined in the previous section.

If there is at least one function call that is not included in the allow list, a tool approval subgraph is invoked.
The tool approval flow is illustrated in the diagram below:

```
mermaid
graph TD;
   start([<p>start</p>]):::first
   approvale(Tools Approval entry Node)
   approvalv(Tools Approval verification Node)
   agent(Agent Node)
   toolsexec(Tools Execution Node)
   end([<p>end</p>]):::last
   start --> agent
   agent --> router{Does any of LLM function calls requires human approval}
   router -->|yes| approvale
   approvale --> toolsvr{Are all function calls valid?}
   toolsvr --> |no| agent
   toolsvr --> |yes| approvalv
   approvalv --> toolsr{Human approval received}
   toolsr --> |no| approvalv
   toolsr --> |human deny| agent
   toolsr --> |human feedback| agent
   toolsr --> |human approve| toolsexec
   router -->|no| toolsexec
   toolsexec --> end
```

At the Tools Approval verification Node, a workflow execution is hibernated to wait for a user's approval, denial, or feedback that instructs agents how to
correct their course.

Milestones

1. All the components implemented and communicating correctly with only a
   trivial workflow implemented.

1. Checkpointing code as well as LangGraph state.
1. Workflow locking in GitLab to ensure only 1 concurrent instance of a workflow.
1. Add more workflows and tools.
1. Ability to resume a workflow.

POC - Demos

1. POC: Solve issue (internal only)
1. POC: Workflow in Workspaces (internal only)
1. POC: Autograph using Docker executor (internal only)
1. POC: Workflows in CI pipelines with timeout and restart (internal only)

SECTION: engineering/architecture/design-documents/duo_workflow/decisions/001_migrate_to_ai_gateway.md

Context

The AI Gateway is a Python-based service that handles LLM interactions over HTTP, primarily acting as a proxy (for authentication, routing, etc.). The Duo Workflow Service is another Python-based service that provides multi-step LLM orchestration (via LangGraph) over gRPC.

Historically, the Duo Workflow Service was developed separately (using gRPC) to allow rapid iteration without integrating into an existing codebase. However, this separation creates overhead in deployments (managing two services specifically for customers using self-hosted models), observability (duplicate logging/tracing), and maintenance (two sets of dependencies). As we want to mature Duo Workflow towards GA and migrate Chat to the Duo Workflow backend, merging Duo Workflow into the AI Gateway can reduce complexity and provide a unified service.

Three options were considered:

1. Option 1: Keep services separate.
2. Option 2: Combine them into one service with two listeners (gRPC & HTTP).
3. Option 3: Combine them into one service with a single listener (e.g., using WebSockets only).

Decision

We decided to combine Duo Workflow Service and AI Gateway into a single repository and Docker image using two listeners (Option 2). Separately we will also be adopting web sockets (Option 3 - see ADR-002) One port will handle the existing HTTP-based AI Gateway traffic, and another port will handle the gRPC-based Duo Workflow traffic. A command-line flag or environment variable can toggle which transports (or both) are enabled at runtime. There will continue to be two services with separate runway deployments for people using SaaS models (.com our Cloud Connected). Self-hosted models customers have two options they can either run a single combined service or two services depending on their own scaling requirements.

Option 3 has been moved to a separate ADR record (ADR-002) as it can be discussed and delivered

independently of merging the two services.

For customers using GitLab hosted models i.e. customers on Gitlab.com or Self-managed customers using GitLab hosted models:

```
mermaid
flowchart LR
  subgraph Runway
    %% First Runway Deployment
    subgraph deploymentHTTP["Deployment 1 - AI Gateway"]
       direction TB
       H["HTTP Listener / AI Gateway"]
    end

    %% Second Runway Deployment
    subgraph deploymentGRPC["Deployment 2 - Duo Workflow"]
       direction TB
       G["gRPC Listener / Duo Workflow"]
    end
  end

  Cloudflare["Cloudflare"] -->|HTTP| H
  Cloudflare["Cloudflare"] -->|gRPC| G
  Clients["External Clients"] -->|HTTP| Cloudflare["Cloudflare"]
  Clients["External Clients"] -->|gRPC| Cloudflare["Cloudflare"]
```

For Self-managed customers using self-hosted models:

```
mermaid
flowchart LR
    subgraph combinedService["Single Docker Image - Combined Service"]
      direction TB
      H["HTTP Listener \(AI Gateway\)"]
      G["gRPC Listener \(Duo Workflow\)"]
    end

    Clients["Direct External Clients"] -->|HTTP| H
    Clients["Direct External Clients"] -->|gRPC| G
```

Note: We will need to update the docs for AI Gateway to show customers how to manage certs for self-hosted deployments of AI Gateway.

Here is example nginx configuration:

```
nginx
upstream aigwgrpcbackend  { server gitlab-ai-gateway:5052; }

server {
```

```
    listen 50052 ssl http2;  gRPC over TLS
    servername ;

    sslcertificate      /etc/nginx/ssl/server.crt;
    sslcertificatekey  /etc/nginx/ssl/server.key;
    sslverifyclient    off;
    sslprotocols        TLSv1.2 TLSv1.3;
    sslciphers          HIGH:!aNULL:!MD5;
    sslpreferserverciphers on;
    sslsessioncache    shared:SSL:10m;
    sslsessiontimeout  10m;

    location / {
        grpcpass grpcs://aigwgrpcbackend;
        grpcreadtimeout    300s;
        grpcconnecttimeout 75s;
        grpcsetheader X-Real-IP        $remoteaddr;
        grpcsetheader X-Forwarded-For  $proxyaddxforwardedfor;
        grpcsetheader X-Forwarded-Host $host;
    }
}
```

Consequences

- Pros
  - Single Docker image for self-hosted deployments simplifies installation and updates.
  - Unified codebase for logs, metrics, secrets management, etc.
  - Minimizes rewriting the existing gRPC components, reducing initial refactoring effort compared to a full WebSocket migration (Option 3).
  - We can scale the two deployments in runway separately since they have different scaling characteristics. e.g. Duo Workflow is more memory intensive since we store a lot of data in workflow state.

- Cons
  - Still requires gRPC support from customers· network configurations. Some firewalls may block HTTP/2 traffic.
  - The hosting platform (Runway) may need to manage two ports/protocols, which can add some complexity.
  - Maintaining both independent servers, even in a single codebase, may still mean there are duplicated efforts where we have differences between gRPC and HTTP
  - There is still an initial migration overhead when migrating the existing code-base
  - We need to decide how to handle reviewer/maintainership when merging two repositories as there is no 100% overlap and Duo Workflow uses different technologies than AI Gateway (LangGraph/gRPC)

Despite these downsides, Option 2 strikes the best balance between maintainability, complexity, and near-term development effort.

SECTION: engineering/architecture/design-documents/duo_workflow/decisions/002_add_websocket_support.md

Context

Initially, the Duo Workflow Service used gRPC over HTTP/2 for streaming. While gRPC provides efficient bi-directional streaming and code generation, it can pose challenges:

- We have first hand experience that enterprise customers are likely to have their security platform such as Netskope or Zscaler configured to block HTTP/2 traffic by default. This leads to added onboarding friction with Duo Workflow, as the security team has to be involved to allow connection.
- Browsers cannot natively use gRPC without a proxy layer (gRPC-Web or Envoy).

Giving the ability for customers to use WebSockets (over HTTP/1.1) can address these issues and unify the transport across all components (client, server, LSP executor). It also enables direct browser-to-service streaming if needed, simplifying real-time feedback loops.

IDE interactions:

mermaid
sequenceDiagram
    participant C as IDE
    participant S as Duo Workflow Service
    participant R as GitLab Rails

    C->>S: Establish WebSocket Connection (Handshake)
    S-->>C: Acknowledge Connection
    C->>S: Send Start Workflow request (Protobuf over WebSocket)
    S-->>S: Start Workflow and call LLM
    S-->>C: Stream LLM tokens to frontend
    C->>S: Next user request/question
    S->>S: LLM requests tool calls
    S-->>C: onToolCalled over Websocket
    alt
        C->>R: Fetch GitLab Data (call tool)
        R-->>C: Tool response
    end
    alt
        C->>C: Read file/write file/run command/run MCP tool
    end
    C-->>S: Tool Response over socket
    S->>S: LLM produces response based on tool result
    S-->>C: Stream LLM tokens

GitLab UI interactions (non-remote execution):

mermaid

```
sequenceDiagram
    participant C as Gitlab UI
    participant S as Duo Workflow Service
    participant R as GitLab Rails

    C->>S: Establish WebSocket Connection (Handshake)
    S-->>C: Acknowledge Connection
    C->>S: Send Start Workflow request (Protobuf over WebSocket)
    S-->>S: Start Workflow and call LLM
    S-->>C: Stream LLM tokens to frontend
    C->>S: Next user request/question
    S->>S: LLM requests tool calls
    S-->>C: onToolCalled over Websocket
    C->>R: Fetch GitLab Data (call tool)
    R-->>C: Tool response
    C-->>S: Tool Response over socket
    S->>S: LLM produces response based on tool result
    S-->>C: Stream LLM tokens
```

Decision

We decided add Web Sockets as an alternative to gRPC for Duo Workflow·s streaming and request/response interactions. Protobuf will be used for serialization over WebSockets where appropriate, preserving type safety.

Websockets have been tested at scale for AI workloads as shown by the following 2 adoptions:

1. Open AI Realtime API
2. Gemini Live API

Consequences

- Pros
  - Single port over standard HTTP/1.1 with WebSocket upgrades is more likely to be firewall-friendly.
  - Browsers can connect directly without needing a separate proxy or gRPC-Web implementation.
  - Unified transport across front-end, back-end, and possibly the LSP executor logic.

- Cons
  - Requires refactoring the existing gRPC-based code, including streaming logic, interceptors, and error handling.
  - gRPC·s built-in features (flow control, code generation) will need to be re-created or replaced with equivalent libraries in the WebSocket ecosystem.
  - Performance differences vs. gRPC in high-load or binary-heavy streaming scenarios may require additional testing or optimization.

The benefit of easier adoption, fewer networking roadblocks, and improved client compatibility outweighs the additional refactoring effort.

SECTION: engineering/architecture/design-documents/duo_workflow/decisions/003_rewrite_workflow_executor_in_typescript.md

Context

The Duo Workflow Service runs in Python, orchestrating multi-step LLM interactions via LangGraph. The executor currently exists as a separate Go binary that the Language Server spawns in order to start executing a workflow. This adds complexity: the LSP is TypeScript-based, but it spawns a Go process for execution. Communication back to the LSP for real-time feedback, streaming and error handling is more limited. There is also duplicated effort in handling local connection and network setup within the Go executor which are already handled in the LSP. Today, remote execution scenarios (running in CI or a container) use the same Go executor.

Proposal

Build a TypeScript Library: Introduce a TypeScript library within the Duo Workflow Service codebase and publish it as an npm module. This library will define the types and interfaces required for the LSP-based executor.
Incorporate Executor Logic into the LSP: Rather than spawning a separate binary, bundle the executor logic directly into the Language Server itself. This allows for:

1. Real-time, bidirectional communication between the Language Server and the Duo Workflow Service.
2. Streamlined error-handling: expired tokens, lint errors, and other relevant messages can be efficiently transmitted back and forth.
3. Better developer ergonomics: everything is in one place (the LSP), rather than relying on a separate compiled binary.

By embedding the executor in the LSP, we remove a layer of indirection, reduce system overhead, and improve real-time feedback mechanisms. This change also lays the foundation for more advanced features·like live streaming of partial results, immediate token refreshes, and better error reporting.

Decision

We decided to adopt a TypeScript executor library for local execution in IDE environments.

- The existing Go executor will remain for CI/remote execution or any environment that requires a lightweight compiled binary in the short term.
- Longer term we decide if we want to have a compiled typescript binary using Node executable, bun or deno or if we'd prefer to maintain both executors.

Consequences

- Pros
  - Direct, in-process communication between the LSP and the executor (no separate binary spawn).

  - Shared type definitions and tooling, reducing friction for developers working on the LSP.
  - Easier incorporation of streaming, incremental feedback, linting, and token refresh flows.
  - Easier distribution: the LSP no longer needs to download the executor binary, but is updated with the LSP itself.

- Cons
  - Requires building and publishing an npm package for the executor logic or merging it directly into the LSP repo.
  - Potential performance and size differences between a compiled TS binary and a Go binary.
  - Maintenance of two executors (TS and Go) in the short term while remote execution flows evolve.

SECTION: engineering/architecture/design-documents/duo_workflow/decisions/004_workhorse_as_a_duo_workflow_service_proxy.md

Context

The Duo Workflow Service is a gRPC server that runs in Python, and orchestrates multi-step LLM interactions via LangGraph. Duo Workflow Executor is a client that runs in a local environment that dynamically sends additional information about the environment to Duo Workflow Service and processes the response. Currently:

- GitLab Rails generates a short-lived token for Duo Workflow Executor to communicate with Duo Workflow Service
- Duo Workflow Executor communicates with Duo Workflow Service directly to send additional information and get the response
- Duo Workflow Executor acts as a proxy when GitLab API requests should be performed: for getting GitLab related information (projects, merge requests), but also for internal purposes to save the current state of a workflow

Problems

- The requests are proxied by Duo Workflow Executor, which causes additional delays and restricts us from sending sensitive information from GitLab Rails to Duo Workflow Service (for example, API keys for self-hosted models)
- The environment that is running Duo Workflow Executor must have access to an additional location (Duo Workflow Service) which creates a burden on individual users to have their system correctly configured for access rather than just an admin if the connection runs via the Workhorse
- The current solution requires the executor to communicate with GRPC, we can add WebSockets as a Fallback, but GRPC might still be preferred for communication between deployed services. More information about gRPC vs Websockets in this ADR

Decision

Use Workhorse as a proxy for Duo Workflow Service. GitLab Workhorse is a smart reverse proxy

for GitLab intended to handle resource-intensive and long-running requests. It can provide a Websocket endpoint to handle communication between Duo Workflow Executor, Duo Workflow Service, and GitLab Rails:

- Duo Workflow Executor opens a Websocket connection with Workhorse when a workflow starts and sends the GitLab private token for authentication and authorization
- Workhorse authorizes the connection by sending an HTTP request to Rails
- Workhorse initiates a gRPC connection with Duo Workflow Service using the auth information provided by GitLab Rails and starts bidirectional communication
- When a gRPC action is received:
  - If it's a run-http request action, Workhorse performs the request directly to Rails using the auth information provided on Websocket initialization
  - Otherwise, Workhorse serializes the action as a JSON and propagates it to Duo Workflow Executor. When the response is received, Workhorse serializes the JSON to gRPC binary data and sends it to Duo Workflow Service

PoCs

The MRs demonstrate communication between Rails, Workhorse, Duo Workflow Service and IDE without issuing a Duo Workflow Service token:

- Rails + Workhorse:
  - Adds a handler for ws route api/v4/ai/duoworkflows/workflows/<id>/connect in Workhorse
  - Adds an authorization handler for api/v4/ai/duoworkflows/workflows/<id>/connect in Rails that passes to Workhorse the information related to Duo Workflow Service
  - In Workhorse, it implements the gRPC communication with Duo Workflow Service and Websockets handling with an executor
- GitLab LSP
  - Initializes a WebSocket connection with Workhorse using the GitLab API private token
  - Processes the Workflow actions in Node Executor

Pros

- The short-lived Duo Workflow token is no longer necessary; a general GitLab personal access token can be used
- Connecting to an additional service via gRPC is no longer necessary; Duo Workflow Executor connects to a GitLab instance using Websockets
- Duo Workflow Executor doesn't proxy GitLab API requests:
  - API requests are executed significantly faster
  - Internal information, like state management, is hidden from a client
  - Sensitive information can be exchanged between Rails and Duo Workflow Service
- Duo Workflow service continues to be a gRPC server only, without providing additional complexity like Websockets connection handling
- Workhorse can act as a deployed Duo Workflow Executor for read-only operations:
  - When a workflow only operates with read-only GitLab API requests, for example, it can be an Agentic Chat in Web UI
  - In this case, a client doesn't need to have any executor logic; it can just react to the results sent by Workhorse via Websockets

Cons

- Performance-related risks. With part of the executor logic running in a deployed environment, the performance issues will be both more visible and more impactful
- The executor logic is now leaked into a third component (Golang Duo Workflow Executor, Node Duo Workflow Executor in GitLab LSP and now part of it in Workhorse), but on the other hand, it simplifies the implementations because they no longer need to communicate with Duo Workflow Service directly
- The API requests performed by Workhorse use the passed private token for auth. If we want to have a composite identify for these calls, we'll have to generate an OAuth token and pass it to Workhorse. It's possible, but it adds an additional responsibility and unnecessary complexity
- Since there needs to be a stateful connection all the way from the Executor to the Duo Workflow Service we would need to drop the gRPC connection every time the websocket connection drops (and vice versa). Without this the websocket reconnects might arrive at the wrong workhorse process which doesn't have the persistent gRPC connection available. Depending on how reliable websockets (without reconnection) are it may mean that workflows get dropped mid way through more often. It also doubles the chance of a dropped connection dropping a workflow. Initially we will always gracefully drop the gRPC connection if websockets drops, the clients will have to resume the workflow on reconnection. If it proves to disconnect too frequently we'll explore using Redis as a reliable transport across any Workhorse server per https://gitlab.com/gitlab-com/content-sites/handbook/-/mergerequests/13685note2515723211

SECTION: engineering/architecture/design-documents/email-ingestion/_index.md

<!-- vale gitlab.CurrentStatus = NO -->

{{< engineering/design-document-header >}}

Summary

GitLab users can submit new issues and comments via email. Administrators configure special mailboxes that GitLab polls on a regular basis and fetches new unread emails. Based on the slug and a hash in the sub-addressing part of the email address, we determine whether this email will file an issue, add a Service Desk issue, or a comment to an existing issue.

Right now emails are ingested by a separate process called mailroom. We would like to stop ingesting emails via mailroom and instead use scheduled Sidekiq jobs to do this directly inside GitLab.

This lays out the foundation for
custom email address ingestion for Service Desk,
detailed health logging and makes it easier to integrate other service provider adapters (for example Gmail via API). We will also reduce the infrastructure setup and maintenance costs for customers on self-managed and make it easier for team members to work with email ingestion in GDK.

Glossary

- Email ingestion: Reading emails from a mailbox via IMAP or an API and forwarding it for processing (for example create an issue or add a comment)
- Sub-addressing: An email address consist of a local part (everything before @) and a domain part. With email sub-addressing you can create unique variations of an email address by adding a + symbol followed by any text to the local part. You can use these sub-addresses to filter, categorize or distinguish between them as all these emails will be delivered to the same mailbox. For example user+subaddress@example.com and user+1@example.com and sub-addresses for user@example.com.
- mailroom: An executable script that spawns a new process for each configured mailbox, reads new emails on a regular basis and forwards the emails to a processing unit.
- incomingemail: An email address that is used for adding comments and issues via email. When you reply on a GitLab notification of an issue comment, this response email will go to the configured incomingemail mailbox, read via mailroom and processed by GitLab. You can also use this address as a Service Desk email address. The configuration is per instance and needs full IMAP or Microsoft Graph API credentials to access the mailbox.
- servicedeskemail: Additional alias email address that is only used for Service Desk. You can also use an address generated from incomingemail to create Service Desk issues.
- deliverymethod: Administrators can define how mailroom forwards fetched emails to GitLab. The legacy and now deprecated approach is called sidekiq, which directly adds a new job to the Redis queue. The current and recommended way is called webhook, which sends a POST request to an internal GitLab API endpoint. This endpoint then adds a new job using the full framework for compressing job data etc. The downside is, that mailroom and GitLab need a shared key file, which might be challenging to distribute in large setups.

Motivation

The current implementation lacks scalability and requires significant infrastructure maintenance. Additionally, there is a lack of proper observability for configuration errors and overall system health. Furthermore, setting up and providing support for multi-node Linux package (Omnibus) installations is challenging, and periodic email ingestion issues necessitate reactive support.

Because we are using a fork of the mailroom gem (gitlab-mailroom), which contains some GitLab specific features that won't be ported upstream, we have a noteable maintenance overhead.

The Service Desk Single-Engineer-Group (SEG) started work on customizable email addresses for Service Desk and released the first iteration in beta in 16.4. As a MVC we introduced a Forwarding & SMTP mode where administrators set up email forwarding from their custom email address to

the projects' incomingmail email address. They also provide SMTP credentials so GitLab can send emails from the custom email address on their behalf. We don't need any additional email ingestion other than the existing mechanics for this approach to work.

As a second iteration we'd like to add Microsoft Graph support for custom email addresses for Service Desk as well. Therefore we need a way to ingest more than the system defined two addresses. We will explore a solution path for Microsoft Graph support where privileged users can connect a custom email account and we can receive messages via a Microsoft Graph webhook (Outlook message). GitLab would need a public endpoint to receive updates on emails. That might not work for Self-managed instances, so we'll need direct email ingestion for Microsoft customers as well. But using the webhook approach could improve performance and efficiency for GitLab SaaS where we potentially have thousands of mailboxes to poll.

Goals

Our goals for this initiative are to enhance the scalability of email ingestion and slim down the infrastructure significantly.

1. This consolidation will eliminate the need for setup for the separate process and pave the way for future initiatives, including direct custom email address ingestion (IMAP & Microsoft Graph), improved health monitoring, data retention (preserving originals), and enhanced processing of attachments within email size limits.
1. Make it easier for team members to develop features with email ingestion. Right now it needs several manual steps.

Non-Goals

This blueprint does not aim to lay out implementation details for all the listed future initiatives. But it will be the foundation for upcoming features (customizable Service Desk email address IMAP/Microsoft Graph, health checks etc.).

We don't include other ingestion methods. We focus on delivering the current set: IMAP and Microsoft Graph API for incomingemail and servicedeskemail.

Current setup

Administrators configure settings (credentials and delivery method) for email mailboxes (for incomingemail and servicedeskemail) in gitlab.rb configuration file. After each change GitLab needs to be reconfigured and restarted to apply the new settings.

We use the separate process mailroom to ingest emails from those mailboxes. mailroom spawns a thread for each configured mailbox and polls those mailboxes every minute. In the meantime the threads are idle. mailroom reads a configuration file that is generated from the settings in gitlab.rb.

mailroom can connect via IMAP and Microsoft Graph, fetch unread emails, and mark them as read or deleted (based on settings). It takes an email and distributes it to its destination via one of the two delivery methods.

webhook delivery method (recommended)

The webhook delivery method is the recommended way to move ingested emails from mailroom to GitLab. mailroom posts the email body and metadata to an internal API endpoint /api/v4/internal/mailroom, that selects the correct handler worker and schedules it for execution.

```
mermaid
flowchart TB
  User --Sends email--> provider[(Email provider mailbox)]
  mailroom --Fetch unread emails via IMAP or Microsoft Graph API--> provider
  mailroom --HTTP POST--> api
  api --adds job for email--> create

  subgraph mailroomprocess[mailroom]
  mailroom[mailroom thread]
  end

  subgraph GitLab
    api[Internal API endpoint]
    create["Sidekiq email handler job
  that create issue/note based
  on email address"]
  end
```

sidekiq delivery method (deprecated since 16.0)

The sidekiq delivery method adds the email body and metadata directly to the Redis queue that Sidekiq uses to manage jobs. It has been deprecated in 16.0 because there is a hard coupling between the delivery method and the Redis configuration. Moreover we cannot use Sidekiq framework optimizations such as job payload compression.

```
mermaid
flowchart TB
  User --Sends email--> provider[(Email provider mailbox)]
  mailroom --Fetch unread emails via IMAP or Microsoft Graph API--> provider

  mailroom --directly writes to Redis queue, which schedules a handler job--> redis[Redis
queue]
  redis --Sidekiq takes job from the queue and executes it--> create

  subgraph mailroomprocess[mailroom]
  mailroom[mailroom thread]
```

```
  end

  subgraph GitLab
    create["Sidekiq email handler job
  that create issue/note based
  on email address"]
  end
```

Proposal

Use Sidekiq jobs to poll mailboxes on a regular basis (every minute, maybe configurable in the future).
Remove all other legacy email ingestion infrastructure.

```
mermaid
flowchart TB
  User --Sends email--> provider[(Email provider mailbox)]
  ingestion --Fetch unread emails via IMAP or Microsoft Graph API--> provider
  controller --"Triggers a job for each mailbox"--> ingestion
  ingestion --Adds a job for each fetched email--> create

  subgraph GitLab
    controller[Scheduled Sidekiq ingestion controller job]
    ingestion[Sidekiq mailbox ingestion job]
    create["Existing Sidekiq email handler jobs
  that create issue/note based
  on email address"]
end
```

1. Use a controller job that is scheduled every minute or every two minutes. This job adds one job for each configured mailbox (incomingemail and servicedeskemail).
1. The concrete ingestion job polls a mailbox (IMAP or Microsoft Graph), downloads unread emails and adds one job for each email that processes the email. We decide based on the used To email address which email handler should be used.
1. The existing email handler jobs try to create an issue, a Service Desk issue or a note on an existing issue/merge request. These handlers are also used by the legacy email ingestion via mailroom.

Sidekiq jobs and job payload size optimizations

We implemented a size limit for Sidekiq jobs and email job payloads (especially emails with attachments)
are likely to pass that bar. We should experiment with the idea of handling email processing directly in the Sidekiq mailbox ingestion job. We could use an ops feature flag
to switch between this mode and a Sidekiq job for each email.

We'd also like to explore a solution path where we only fetch the message ids
and then download the complete messages in child jobs (filter by UID range for example).

For example we poll a mailbox and fetch a list of message ids. Then we create a new job for every 25 (or n) emails that takes the message ids or the range as an argument. These jobs will then download the entire messages and synchronously add issues or replies. If the number of emails is below 25, we could even handle the emails directly in the current job to save resources. This will allow us to eliminate the job payload size as the limiting factor for the size of emails. The disadvantage is that we need to make two calls to the IMAP server instead of one (n+1).

Execution plan

1. Add deprecation for mailroom email ingestion.
1. Strip out connection-specific logic from gitlab-mailroom gem, into a new separate gem. mailroom and other clients could use our work here. Right now we support IMAP and Microsoft Graph API connections.
1. Add new jobs (set idempotency and de-duplication flags to avoid a huge backlog of jobs if Sidekiq isn't running).
1. Add a setting (gitlab.rb) that enables email ingestion with Sidekiq jobs inside GitLab. We need to set mailroom['enabled'] = false in gitlab.rb to disable mailroom email ingestion. Maybe additionally add a feature flag.
1. Use on gitlab.com before general availability, but allow self-managed to try it out in beta.
1. Once rolled out in general availability and when removal has been scheduled, remove the dependency to gitlab-mailroom entirely, remove the internal API endpoint api/internal/mailroom, remove mailroom.yml dynamically generated static configuration file for mailroom and other configuration and binaries.

Change management

We decided to deprecate the sidekiq delivery method for mailroom in GitLab 16.0 and scheduled it for removal in GitLab 18.0.
We can only remove the sidekiq delivery method after this blueprint has been implemented and our customers can use the new email ingestion in general availability.

We should then schedule mailroom for removal (GitLab 17.0 or later). This will be a breaking change. We could make the new email ingestion the default beforehand, so self-managed customers wouldn't need to take action.

Alternative Solutions

Do nothing

The current setup limits us and only allows to fetch two email addresses. To publish Service Desk custom email addresses with IMAP or API integration we would need to deliver the same architecture as described above. Because of that we should act now and include general email ingestion for incomingemail and servicedeskemail first and remove the infrastructure overhead.

Additional resources

- Meta issue for this design document

Timeline

- 2023-09-26: The initial version of the blueprint has been merged.

SECTION: engineering/architecture/design-documents/encryption_keys_rotation/_index.md

{{< engineering/design-document-header >}}

Introduction

We need a solution to rotate GitLab's encryption keys (used to protect sensitive data at rest) without requiring system downtime, addressing a significant security and operational risk for both our customers and GitLab.com. The proposed solution enables zero-downtime key rotation through support for multiple concurrent keys, automated background re-encryption, and a deployment workflow optimized for multi-node installations, allowing organizations to maintain security best practices without service interruption.

Objectives

This design document addresses a big inconvenience in GitLab's current encryption key management system.
Currently, GitLab's encryption keys, which protect sensitive data at rest in the database, cannot be rotated without
taking the entire system offline. This limitation poses significant business risks:

1. In the event of a key compromise, customers would face service disruption during key rotation
2. For GitLab.com and large enterprise deployments, the required downtime makes regular key rotation practically
   impossible
3. With the expansion of GitLab's Dedicated and Cells infrastructure, the risk of key compromise increases, as does
   the operational complexity of key management across multiple instances
4. In Cells 1.0, all cells will use the same secrets,
   wich further increases the risk and impact of any key exfiltration.

The proposed solution enables zero-downtime encryption key rotation through:

- Support for multiple concurrent encryption keys
- Automated background re-encryption of data
- Clear visibility into key usage and rotation progress through a new admin interface
- A deployment workflow optimized for multi-node installations

Key business benefits:

- Eliminates service disruption during security-critical key rotations
- Enables compliance with security policies requiring regular key rotation
- Reduces operational risk in multi-instance deployments, especially in the context of Cells

- Provides foundation for secure data movement between GitLab instances, especially when using the Cells Org Mover

The implementation is planned across seven iterations, focusing on maintaining system stability while introducing this
capability. This project directly supports GitLab's scaling initiatives and enterprise security requirements.

Overview

In the Rails monolith:

- Introduce an always-running background process (with automated throttling to avoid degrading database performance) to take
  care of progressive re-encryption while GitLab stays online. The process would look up and re-encrypt any data
  encrypted with a legacy encryption key.

For infrastructure operators (self-managed administrators, GitLab.com SREs, Dedicated SREs):

- Provide the ability to introduce a new encryption key that will replace the previous one for encrypting data.
  The new key as well as all previous keys are used for decryption.
- Introduces a deployment workflow for multi-node installations where the new encryption key is first introduced at the
  head of the keys array (so that it's first only used for decryption), then when the new key is deployed to all nodes,
  it would need to be moved to the tail of the keys array, so that it's used for encryption from now on.
  This is important in the scenario where an infrastructure operator adds a new key to config/secrets.yml, and then kicks off a
  new deployment. During the deployment phase, some of the pods/VMs will not have the new key. It's critical that the
  new pods/VMs don't start re-encrypting using the new key until the deployment is completed.
  When the deployment completes successfully, all pods/VMs now have the new key and the administrator can move the key
  to be last in the keys array so that it becomes the current encryption key.

For instance administrators (self-managed administrators, GitLab.com SREs, Dedicated customers):

- Provide a new Admin page to list keys, including their status and usage, as well as monitor the progress of the
  re-encryption of data encrypted with legacy keys.

Non-goals

This blueprint does not cover the following:

- Introduce scripts to re-encrypt all the data while GitLab is offline. While there might be

customer use-cases for
this, we won't implement this initially.
- Other keys & secrets such as secretkeybase, otpkeybase, openidconnectsigningkey, and
encryptedsettingskeybase, but ideally it should describe a solution that's generic enough to be applied to other
secrets without too much changes in the future.
- Possibility to rotate the encryption keys from the Admin UI (this would require writing to the config/secrets.yml
at runtime, which is impossible with most deployment strategies).
- Possibility to pull encryption keys from an external secrets manager (e.g. GCP and AWS KMS).
While this is related and more secure, it should be solved with a dedicated proposal.
- Decision to use envelope encryption or not.
While envelope encryption has many benefits and is supported natively by Active Record Encryption, the decision to
use it is independent from this proposal, and should be solved with a dedicated proposal.

Pre-requisites

The idea is based on 2 pre-requisites:

1. Support for multiple encryption keys: this allows online rotation of the secret
1. Ability to know what key was used to encrypt an attribute: this allows to re-encrypt data encrypted with a legacy key

Rotation workflow

1. When a key needs to be rotated, just add a new one to the tail of the
dbkeybase / activerecordencryptionprimarykey / activerecordencryptiondeterministickey arrays in
config/secrets.yml, and restart GitLab. In multi-nodes installations, the new key deployment should happen in two
phases: First add the key at the head of the keys array and wait for it to be deployed everywhere; then move the key to
the tail of the keys array and start a new deployment.
1. Once deployed at the tail of the keys array, the new key becomes the current encryption key.
1. The decryption process uses the key that was used to encrypt the data.
In the case the encryption key ID isn't stored alongside the encrypted data, the decryption process tries
each key (in the order they appear in the key arrays), until it can decrypt the data.
That way, there's no need to bring GitLab down to mass-re-encrypt all data.
1. A background process continuously runs to re-encrypt any data that was encrypted with the non-current encryption key.
The whole re-encryption process would likely take a long time on big instances (e.g. GitLab.com), but as long as we
have a limiting/throttling mechanism in place, it shouldn't impact the database stability.
The background process becomes a no-op as soon as all the data is re-encrypted with the current encryption key.
1. A new dedicated admin page allows to monitor the encryption key usage:
- What percentage of data is encrypted with each active encryption/decryption keys?
- What's the expected ETA for everything to be re-encrypted with the current encryption key?

  - What keys can be removed from config/secrets.yml (i.e. no data is encrypted with this key anymore)?

"Encryption keys" admin page mockup

Decisions

Implementation Details

Keys tracking

Keys lifecycle information will be stored in a new encryptionkeys table, including:

- Key type keytype, an enum:
  - dbkeybase
  - dbkeybase32
  - dbkeybasetruncated
  - activerecordencryptionprimarykey
  - activerecordencryptiondeterministickey
- Key fingerprint (4 hex chars), (inspired by <https://github.com/rails/rails/blob/v7.0.8.6/activerecord/lib/activerecord/encryption/key.rb L24>).
- Key creation time createdat: the first time it appeared in the config/secrets.yml file.
- Key deletion time removedat: the time when the key disappeared from the config/secrets.yml file.

Keeping track of this information in the database provides the following capabilities:

- See the current encryption and decryption keys
- Keep track of keys that were used in the past

Later on, we could also keep statistics about keys usage in a separate table.

Note: The actual keys used to encrypt/decrypt data still come from the config/secrets.yml file.

During initialization

During initialization, if a new key is discovered in config/secrets.yml, the following 3 steps happens:

1. If the new key's fingerprint collides with an existing non-removed key fingerprint, the application fail to start
   as it would introduce ambiguity when picking the decryption key at decryption time.
   In that case, the key should be removed and replaced with another key.
   Note that a rake task will be provided to perform this collision check before the key is actually added to config/secrets.yml.
1. A record for the new key is created in the encryptionkeys table.
1. If a key is present in encryptionkeys but not in config/secrets.yml anymore, the record's

removedat is set.
  If the key was still in use (based on the usage data we will regularly compute), an error is raised to prevent the
  application from starting without the key.

Encryption key selection

We'll introduce a KeyProviderService that will abstract the encryption/decryption keys selection so that's it's
framework-agnostic (to ease the transition from legacy encryption framework to Active Record Encryption).

Architecture

mermaid
classDiagram
  class KeyInitializer {
    +secretsfilekeyfingerprintsperkeytype()
    +populate!()
  }

  class KeyProviderService {
    +keyprovider()
    +encryptionkey()
    +decryptionkeys()
  }

  class EnvelopeEncryptionKeyProvider {
    +encryptionkey()
    +decryptionkeys()
  }

  class DerivedSecretKeyProvider {
    +encryptionkey()
    +decryptionkeys()
  }

  class KeyProvider {
    +encryptionkey()
    +decryptionkeys()
  }

  class EncryptionKey {
    +id
    +keytype
  }

  KeyInitializer --> EncryptionKey : populates
  KeyInitializer --> KeyProviderService : fetches keys fingerprints
  KeyProviderService --> EnvelopeEncryptionKeyProvider : instantiates

```
KeyProviderService --> KeyProvider : instantiates
EnvelopeEncryptionKeyProvider --> DerivedSecretKeyProvider : instantiates
```

Changes required for attrencrypted

The attrencrypted gem supports dynamic key by passing a method name as the key: option, e.g.

ruby
attrencrypted :email, key: :dbkeybase32

We're taking advantage of that so that the key(s) used for encryption/decryption are retrieved from
Gitlab::Database::Encryption::KeyProviderService.

See the PoC code at <https://gitlab.com/gitlab-org/gitlab/-/mergerequests/177748/diffs?commitid=828f2470e5d034e77b7c094952ff2d7676cf962f5d31008bc68bfcfa387a3338a808f53c51a6ad5>.

Changes required for TokenAuthenticatable

Changes are similar to what's done for attrencrypted.

See the PoC code at <https://gitlab.com/gitlab-org/gitlab/-/mergerequests/177748/diffs?commitid=828f2470e5d034e77b7c094952ff2d7676cf962fa99cfc117c9fe8408818387e8197ef3186848efe>.

Implementation of "Encryption keys" admin page

See the PoC code at <https://gitlab.com/gitlab-org/gitlab/-/mergerequests/177838/diffs?commitid=a5cafd65bb706fd1ad2784d7a35592443f02980e>.

Background re-encryption process

Following is a naive implementation of what the background re-encryption process would roughly do.

ruby
```
ActiveRecord::Encryption
def reencryptsactiverecordencryption(model, attributes, keyfingerprint)
  attributes.each do |attr|
    model.where("NOT ({attr}->'h'->'i') ? :value", value:
::Base64.strictencode64(keyfingerprint))
      .findinbatches do |batch|
      batch.each do |record|
        record.encrypt  this forces the re-encryption of all encrypted attribute
      end
    end
  end
```

```
end

def reencryptslegacyencryption(model, attributes, encryptionkeyid)
  model.where.not(encryptionkeyid: encryptionkeyid).findinbatches do |batch|
    batch.each do |record|
      attributes.each do |attr|
        record.publicsend(:"{attr}=", record.publicsend(attr))
      end
      record.save!
    end
  end
end

ApplicationRecord.descendants.select { |d| d.deterministicencryptedattributes.present? }.each
do |model|
  reencryptsactiverecordencryption(
    model,
    model.deterministicencryptedattributes,
    Gitlab::Database::Encryption::KeyProviderService.new(:activerecordencryptiondeterministicke
y).encryptionkey.id
  )
end

ApplicationRecord.descendants.select { |d| d.encryptedattributes.present? }.each do |model|
  reencryptsactiverecordencryption(
    model,
    model.encryptedattributes,
    Gitlab::Database::Encryption::KeyProviderService.new(:activerecordencryptionprimarykey).enc
ryptionkey.id
  )
end

 TokenAuthenticatable
encryptionkey = EncryptionKey.findbyfingerprint(Gitlab::CryptoHelper.encryptionkey.id)

ApplicationRecord.descendants.select { |d| d.include?(TokenAuthenticatable) &&
d.encryptedtokenauthenticatablefields.present? }.each do |model|
  encryptedfields = model.encryptedtokenauthenticatablefields

  reencryptslegacyencryption(model, encryptedfields, encryptionkey.id)
end

 attrencrypted
ApplicationRecord.descendants.select { |d| d.attrencryptedattributes.present? }.each do |model|
  encryptedfields = model.attrencryptedattributes
  keytype = encryptedfields[encryptedfields.keys.first][:key]
  currentkeyfingerprint =
Gitlab::Database::Encryption::KeyProviderService.new(keytype).encryptionkey.id

  reencryptslegacyencryption(model, encryptedfields,
```

EncryptionKey.findbyfingerprint(currentkeyfingerprint).id)
end

The final implementation could build upon
the Background migration framework, which includes a throttling mechanism.

Also, using the Background migration framework, we could have one background migration per table to be re-encrypted,
and report its progress
directly in the admin UI.

Data encrypted through ActiveRecord::Encryption

The ActiveRecord::Encryption framework already fulfills the pre-requisites (except for rotating deterministic keys,
but we might work around that, or even implement proper support for it).

Data encrypted through attrencrypted and TokenAuthenticatable

The current plan is to migrate all the usage of attrencrypted and TokenAuthenticatable to ActiveRecord::Encryption.

Alternatively, since attrencrypted and TokenAuthenticatable don't store the fingerprint of the key used to encrypt an attribute,
we could introduce a new encryptionkeyid column referencing the EncryptionKeyid column to tables that include
encrypted columns.

A single encryptionkeyid column per table is enough since the same key is used to encrypt all encrypted attributes
for a given record.

Once introduced, a post-deploy migration should populate all rows with the current encryption key ID.

The implementation of attrencrypted and TokenAuthenticatable would need to be modified to populate the
encryptionkeyid attribute.

Other usages of dbkeybase

There are few places where the dbkeybase secrets are used (mostly in JWT generation):

- Auth::DependencyProxyAuthenticationServicesecret in
app/services/auth/dependencyproxyauthenticationservice.rb
- Gitlab::Geo::Oauth::LogoutStatewithcipher in ee/lib/gitlab/geo/oauth/logoutstate.rb
  - Note that in Gitlab::Geo::Oauth::LoginStatekey, we use
Gitlab::Application.credentials.secretkeybase...
- Gitlab::ConanTokensecret in lib/gitlab/conantoken.rb

- Gitlab::JWTTokensecret in lib/gitlab/jwttoken.rb
- Gitlab::LfsToken::HMACTokensecret in lib/gitlab/lfstoken.rb

In all these cases, we should probably follow the general practice:

- Encrypt with the current active key
- Try to decrypt with all keys until one works

Rotation of deterministic key

Deterministic encryption allows to query a table for a specific column value (e.g. personal access tokens are currently
queried by their digest, but we should migrate them to be encrypted instead so that we can rotate the key without
invalidating all the tokens).

ActiveRecord::Encryption doesn't support deterministic keys rotation at the moment, support for it should be
implemented either in GitLab, or in Rails directly.

That said, we might be able to work around this limitation by specifying a custom key provider so that under the hood
it uses the logic from DerivedSecretKeyProvider but with the deterministic: true option which makes the encryption
process generate the initialization vector based on the encrypted content instead of being random, i.e.

ruby
encrypts :attr, deterministic: true, keyprovider:
Gitlab::Database::Encryption::KeyProviderService.new(:activerecordencryptiondeterministickey)

Proof of Concept merge requests

Multiple encryption key support

The following merge request shows that support of multiple encryption keys is possible today for both attrencrypted
and TokenAuthenticatable: <https://gitlab.com/gitlab-org/gitlab/-/mergerequests/177748>

What's missing from this PoC is the second pre-requisite: the ability to know
what key was used to encrypt an attribute. It's possible to add support for this with the introduction of a new
encryptionkeyid column per table.

Encryption keys services and admin UI

The following merge request implements the basis for encryption keys services and admin UI:
<https://gitlab.com/gitlab-org/gitlab/-/mergerequests/177838>

What's missing from this PoC is:

- the automated background process to re-encrypt legacy-key-encoded data
- key usage statistics in the admin

Iteration plan

Iteration 1: Introduce encryption key services and support multiple dbkeybase

1. Introduce KeyProviderService to abstract encryptionkey and decryptionkeys similarly to how ActiveRecord::Encryption does.
1. Implement support for multiple keys in attrencrypted and TokenAuthenticatable but raise if more than one key is
   defined (until iteration 4 is done).
1. Implement support for multiple keys in other usages of dbkeybase

Epic: <https://gitlab.com/groups/gitlab-org/-/epics/17142>

Iteration 2: Migrate encrypted attributes to Active Record Encryption

1. Migrate all models that use attrencrypted and TokenAuthenticatable to ActiveRecord::Encryption

Epic: <https://gitlab.com/groups/gitlab-org/-/epics/16793>

Iteration 3: EncryptionKey model implementation

1. Implement the keys tracking system in the database
   - Create a new encryptionkeys table to store keys information (id, fingerprint, status, timestamps)
   - Add collision detection mechanism

Epic: <https://gitlab.com/groups/gitlab-org/-/epics/17143>

Iteration 4: Encryption keys initializer

1. Implement initializer calling KeyInitializer.populate! to read keys from config/secrets.yml and populate/update the database.

Epic: <https://gitlab.com/groups/gitlab-org/-/epics/17144>

Iteration 5: Encryption keys admin interface

1. Create the admin interface for keys tracking
   - Develop the UI for viewing key status, and usage statistics

Epic: <https://gitlab.com/groups/gitlab-org/-/epics/17147>

Iteration 6: Re-encryption Process

1. Implement background re-encryption process
   - Build on top of background migration framework
   - Ensure database load is under control during re-encryption
   - Support ActiveRecord::Encryption, attrencrypted, and TokenAuthenticatable
1. Develop progress tracking for re-encryption
   - Add database columns to track re-encryption progress
   - Update admin interface to display re-encryption status

Epic: <https://gitlab.com/groups/gitlab-org/-/epics/17145>

Iteration 7: Additional tooling

1. Create rake task to detect key collision in advance
1. Develop safeguards against accidental key deletion

Epic: <https://gitlab.com/groups/gitlab-org/-/epics/17146>

References

- <https://gitlab.com/gitlab-org/gitlab/-/issues/25332>
- <https://gitlab.com/gitlab-org/gitlab/-/issues/26243>
- <https://gitlab.com/groups/gitlab-org/-/epics/10193>
- <https://gitlab.com/gitlab-com/gl-infra/gitlab-dedicated/team/-/issues/594> (confidential)
- <https://gitlab.com/gitlab-org/gitlab/-/issues/228663> (confidential)
- <https://gitlab.com/gitlab-com/gl-security/security-department-meta/-/issues/756> (confidential)
- <https://gitlab.com/gitlab-org/gitlab/-/issues/244855> (confidential)
- <https://gitlab.com/groups/gitlab-com/gl-infra/-/epics/443> (confidential)
- <https://gitlab.com/gitlab-com/gl-infra/production-engineering/-/issues/12927> (confidential)

Who

DRIs:

<!-- vale gitlab.Spelling = NO -->

Role	Who
Author	Rémy Coutable, Principal Engineer

<!-- vale gitlab.Spelling = YES -->

SECTION: engineering/architecture/design-documents/epss/_index.md

<!--
Before you start:

- Copy this file to a sub-directory and call it index.md for it to appear in

  the blueprint directory.
- Remove comment blocks for sections you've filled in.
  When your blueprint ready for review, all of these comment blocks should be
  removed.

To get started with a blueprint you can use this template to inform you about what you may want to document in it at the beginning. This content will change / evolve as you move forward with the proposal. You are not constrained by the content in this template. If you have a good idea about what should be in your blueprint, you can ignore the template, but if you don't know yet what should be in it, this template might be handy.

- Fill out this file as best you can. At minimum, you should fill in the
  "Summary", and "Motivation" sections. These can be brief and may be a copy
  of issue or epic descriptions if the initiative is already on Product's
  roadmap.
- Create a MR for this blueprint. Assign it to an Architecture Evolution
  Coach (i.e. a Principal+ engineer).
- Merge early and iterate. Avoid getting hung up on specific details and
  instead aim to get the goals of the blueprint